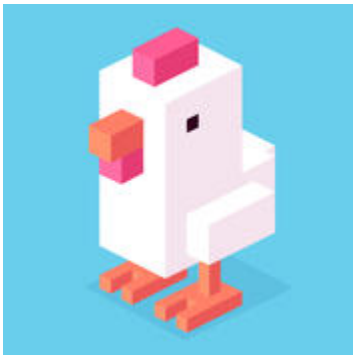
Crossy Road 3D



Author(s):

James Dwyer, dwyerj5@mymacewan.ca

Zhire Pelayo, pelayoz@mymacewan.ca

Introduction

Crossy Road is a 3D movement platformer where the player is tasked with navigating the ladybug across an endless series of roads as far as possible. Players must avoid getting hit by traffic; otherwise, the game ends.

Additionally, players may collect coins randomly scattered throughout the roads. This gives players a sense of accomplishment and incentive to play for longer.

Our game is inspired by a mobile game called “Crossy Road.” The game plays similarly, where the player’s goal is to control a chicken without getting hit by the moving cars.

Challenges

- Smooth movement implementation
    - We wanted the objects on the scene to move smoothly, without it looking unnatural and janky.
- Correct and deletion of objects or terrain
    - The game was planned to be procedurally generated, as we didn’t want to be limited by the objects that we placed using Zach’s engine.
- Correct and proper collision detection
    - Collisions needed to be implemented because there are a lot of objects on the scene, such as the cars, walls, and coins.
- UI and SFX implementation
    - UI and SFX are additional topics that were not covered in the lectures, so we needed to do a bit of research on how to implement these features properly.

Methods

Summary of functionality

- Transformation and Rotation - objects translate/rotate along the x-, y-, and z-axis.

```
switch (e.key) {
    case "a":

        this.cube.translate(vec3.fromValues(-0.5, 0, 0));
        break;



    case "d":

        this.cube.translate(vec3.fromValues(0.5, 0, 0));
        break;


    case "w":

        this.cube.translate(vec3.fromValues(0, 0, -0.5));
        break;


    case "s":

        this.cube.translate(vec3.fromValues(0, 0, 0.5));
        break;

    case " ":
        if (!this.firstPerson) {
            this.setCameraToPlayerView();
            this.firstPerson = true;
            break
        } else {
            this.setCameraToThirdPerson();
            this.firstPerson = false;
        }


    default:
        break;
}
```

* Movement implementation

```
// sets all coin objects to rotate
this.state.objects.forEach(object => {
    if (object.isCoin) {
        object.rotate('y', deltaTime * 2.0);
    }
});
```

* Code snippet of coin rotating

- Blinn-Phong shading model - used to calculate the ambient, diffuse, and specular lighting.

```
// Ambient
vec3 ambient = (ambientVal * light.colour) * light.strength;

// Diffuse
float NdotL = max(dot(normal, lightDirection), 0.0);
vec3 diffuse = (diffuseVal * light.colour) * NdotL;

//Specular
//vec3 nCameraPosition = normalize(oCameraPosition); // Normalize the camera position
vec3 V = normalize(oCameraPosition - oFragPosition);
vec3 H = normalize(V + lightDirection); // H = V + L normalized

vec3 specular = vec3(0.0,0.0,0.0);
if (NdotL > 0.0f)
{
    float NDotH = max(dot(normal, H), 0.0);
    float totalNVal = nVal * nValueConstant;
    float NHPow = pow(NDotH, totalNVal); // (N dot H)^n
    specular = (specularVal * light.colour) * NHPow;
}
return ambient + diffuse + specular;
```

* Blinn-Phong implementation

- Collision using bounding spheres

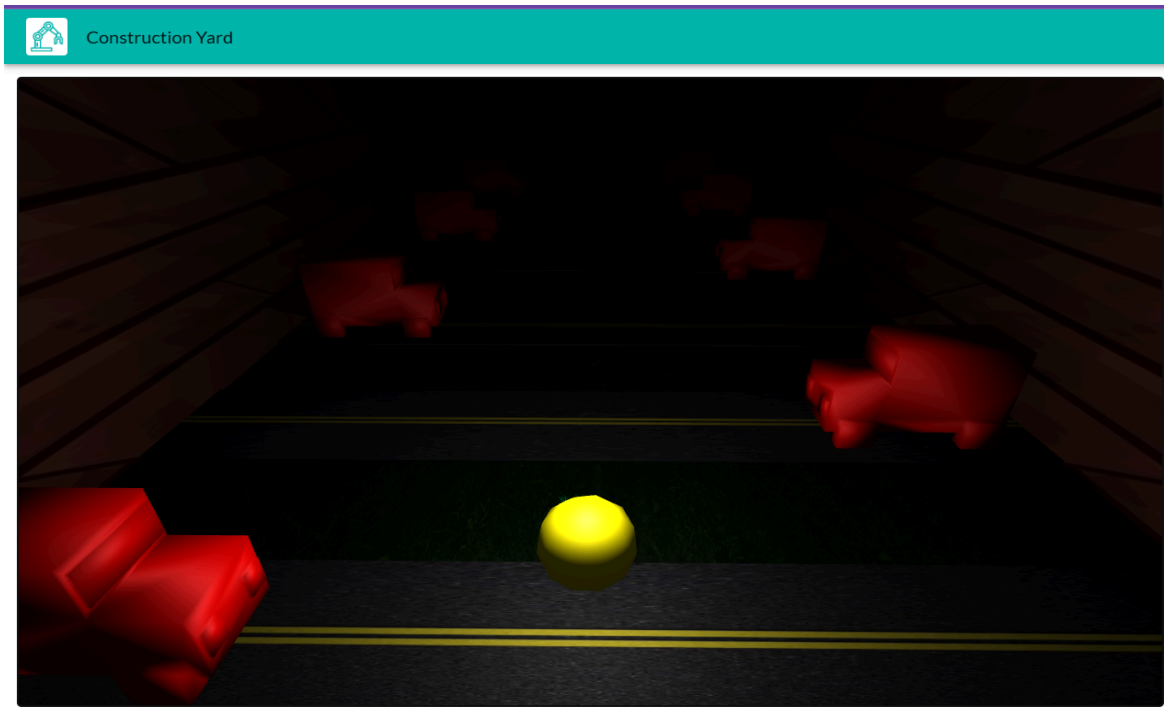
```
checkCollision(object) {
    // loop over all the other collidable objects
    this.collidableObjects.forEach(otherObject => {
        // probably don't need to collide with ourselves
        if (object.name === otherObject.name) {
            return;
        }
        let position1 = vec3.create();
        vec3.transformMat4(position1, object.model.position, object.modelMatrix);

        let position2 = vec3.create();
        vec3.transformMat4(position2, otherObject.model.position, otherObject.modelMatrix);

        let distance = vec3.distance(position1, position2);
        // check if the object isnt colliding with itself and that the distance is less than the 2 radius values combined
        if (otherObject.name !== object.name && (distance < (object.collider.radius + otherObject.collider.radius))) {
            object.collider.onCollide(otherObject);
        }
```

*Code Snippet for checking if a collision has occurred”

- **Playing field/terrain**
    - Made from Zach’s engine and loaded into the graphics pipeline using cube objects, as well as texture assets loaded onto them



“Taken from Zach’s construction yard engine”.

- **Main Player**
  - Main player is translated on a 2d plane of X and Z
  - Using the screenshot on the transformations
- **Add objects and interaction with the main player**
  - Coin objects are created procedurally onto the roads and are given a sphere collision to interact with the player

```
async spawnCoinsOnLane(index){
    const baseZ = -(index * 8);
    const startX = this.getRandomInt(12);

    const coin = await spawnObject({
        name: `coin_${index}_${Math.random() * 9999}`,
        type: "mesh",
        fileName: "icosphere2s.obj",
        position: vec3.fromValues(startX, 1.5, baseZ),

        scale: vec3.fromValues(1, 1, 1),
        material: {
            diffuse: vec3.fromValues(0.9215686321258545, 0.8627451062202454, 0.019607843831181526),
            diffuseTexture: { name: "istockphoto-533725713-612x612-removebg-preview.png" },
            normalTexture: { name: "istockphoto-533725713-612x612-removebg-preview.png" }
        }

    }, this.state);

    this.createSphereCollider(coin, 1.6)
}
```

*Code snippet of spawning the coins on the lane”

- **Non-player Character**
  - Car Objects are added procedurally onto their lanes and are given a bounding sphere collision so that they can interact with the player

```
async spawnCarsOnLane(index) {
    const baseZ = -(index * 8);

    const directionRight = Math.random() < 0.5;
    const startX = directionRight ? -12 : 12;


    const car = await spawnObject({
        name: `Car_${index}_${Math.random() * 9999}`,
        type: "mesh",
        fileName: "car.obj",
        position: vec3.fromValues(startX, 0.51, baseZ),

        scale: vec3.fromValues(1, 1, 1),
        material: {
            diffuse: vec3.fromValues(1, 0, 0),
        }

    }, this.state);
    if (directionRight) {
        car.rotate('y', Math.PI / 2);
    } else {
        car.rotate('y', -Math.PI / 2);
    }

    console.log(`Car ${car.name} model matrix:`, car.modelMatrix);
    this.createSphereCollider(car, 1.75);


}
```

*Code snippet of spawning the car objects on a lane”

- **Change of View**
  - Camera position is changed when pressing the “Space” key, and it changes depending on whether the player is in first-person or third-person mode

```
case " ":
    if (!this.firstPerson) {
        this.setCameraToPlayerView();
        this.firstPerson = true;
        break
    } else {
        this.setCameraToThirdPerson();
        this.firstPerson = false;
    }
```

```
setCameraToPlayerView() {

    const cam = this.state.camera;
    const p = this.cube.model.position;

    // Move camera directly to player location
    cam.position = vec3.fromValues(p[0], p[1] + 1, p[2] - 1.7);

    // Look the same way the player is facing
    cam.front = vec3.fromValues(0, 0, -1);
    console.log("Camera moved to player view");
}
setCameraToThirdPerson() {
    const cam = this.state.camera;
    const p = this.cube.model.position;
    cam.position = vec3.fromValues(
        p[0],
        p[1] + 18,
        p[2] + 1
    );
    let toPlayer = vec3.fromValues(
        p[0] - cam.position[0],
        p[1] - cam.position[1],
        p[2] - cam.position[2]
    );

    cam.front = toPlayer;
    console.log("Camera move to third person")
}
```

Code snippet of setting the Camera positioning

Link to Theory

- Shading and light models - the light is done in the fragment shader, and implements the Phong or Blinn-Phong shading model.
- Object modelling was done from Zach's Engine using cubes, spheres, and meshes, combined with textures
- Collision is taken from the theory where the distance between the centers of the spheres is less than the sum of their radii
- Textures are done in the fragment shader; there were no special textures, like bump maps used

```
if (samplerExists == 1) {
    vec3 textureColour = texture(uTexture, oUV).rgb;
    fragColor = vec4(total * textureColour, 1.0);
```

- Visibility is done by having the front of the camera always be positioned to be at the player if in third person and always in front of the player if in first person, using the coordinates of the player matrix
- For transparency, we did not including any transparent components, however if we were to, we would be sorting the objects and using the depth buffer and changing the alpha values depending on their depth.

Implementation Details

- Player Movement - movement is implemented through the use of transformations, where the player object is translated along the x- and z-axis.
- Coins - implemented using a mixture of transformation, rotation, and collision.
- Change of view - the change in perspective is done with the use of transformation for the camera.
- Procedural Generation - Procedural generation is done by using Math.Random to have a 50/50 chance of spawning a "road" lane or a "grass" lane, and if a road lane is determined, it creates it using road material and name and the position is based on the player's position. The road is then pushed into a lanes list where it can be added to or deleted from. If the lane is a "road" lane, then it will also spawn a truck on either side of the road, as well as a coin randomly within the dimensions of the road
- Cars - Cars are spawned in with a sphere collider that, when hit by the player's sphere collider, will restart the game
- UI - the UI text and button are implemented in the HTML files in their own sections separated by the tag. These tags are then called by "getElementById" in the game.js file.
- SFX - sound effects are first downloaded and added to the assets folder. They are then initialized in the game.js file and called by the play() function.

Video Demo

Crossy Road demo

GitHub Link

Final Project GitHub Link

Analysis and Discussion

- What Worked:
    - Our camera swapping worked how we wanted, with both first-person and third-person views
    - Our Collision worked well without many issues
    - Our lane procedural generation turned out better than initially expected
    - Textures loaded properly
- What didn't work:
    - Our movement is not what we initially thought, and is quite "clunky" and imperfect
    - There is no time or motivation component, making the player constantly move forward in fear of losing, compared to Crossy Road
- What we learned:

  This project helped us understand what goes on behind how methods such as transformation, rotation, lighting, and textures are implemented. Nowadays, game engines such as Zach's make them easy to implement, but this course gave more insights into just how computer graphics is used to create and develop computer games.

- If we continue this project:
    - We would add more features, such as more obstacles on the road, similar to how Crossy Road has rocks that prevent the player from simply going forward.
    - We would like to have a smoother and improved motion to create the illusion of animation, compared to its current implementation.
    - We would like some more improved dynamic procedural generation, such as varying car speeds and improved randomness in road spawning.
    - The camera being able to "overtake" the player, similar to how Crossy Road has a time component to it.
- Contributions:
    - Zhire - I mainly worked on the scene setup, shading, and textures of the objects. I also implemented the logic behind picking up the coins, as well as UI and SFX-related features.
    - James - I worked on the core of the games mechanics including the procedural generation, Car Spawning, car collisions, coins spawning, wall collisions, camera movement, camera swapping.